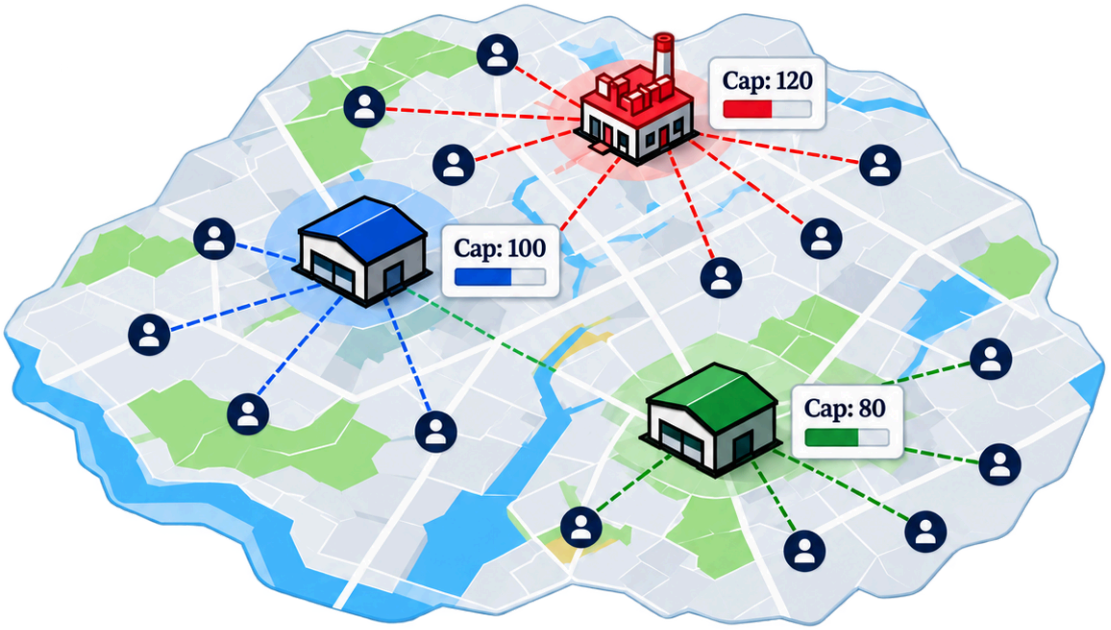
# ACIT 4610 - Assignment 2: Multiobjective CFLP using VEGA and NSGA-II, Group-1

Niklas Goldhan, Alexander Hoem, Noah Øfsti, Michael Huse

https://github.com/NiklasGoldhan/ACIT4610_assignment2



# Introduction

In this assignment we are solving a Multi-Objective Capacitated Facility Problem using two distinct multi-objective evolutionary algorithms, VEGA and NSGA-2. Our problem has a number of facilities with varying capacity and a number of customers with demands that need to be satisfied. The opening of a facility and assigning of a customer to a facility incurs costs specified in each problem instance. Our goal is to find trade-off solutions minimizing the following two objectives:

1) Total cost of facility openings.

$$minf_1 = \sum_i F_i y_i$$

2) Total cost of customer allocations.

$$minf_2 = \sum_i \sum_j C_{ij} x_{ij}$$

Here $y$ and $x$ are binary values specifying which values have been selected and should be included in the summation.

The data in the OR library gives an opening cost and a capacity for each facility, these opening costs are the costs relevant for objective 1. For each customer it gives a demand and it's assignment cost for each of the facilities, it is this assignment cost we look at for objective 2.

In addition, there are three constraints imposed on the problem:

1) Every customer *j* is assigned exactly once.

$$\sum_i x_{ij} = 1 \qquad \forall j$$

2) Assignment only to an opened facility.

$$x_{ij} \leq y_i$$

3) Facility capacity must not be exceeded for any facility $i$.

$$\sum_j d_j x_{ij} \leq S_i y_i \qquad \forall i$$

# Base algorithm

## Solution representation

A list of integers with length `customer_count`, where index `j` represents customer `j` and the stored value is the assigned warehouse `i` with warehouses `0 … n`. This representation makes sure that every customer is assigned to exactly one facility.

## Feasibility Repair

Procedure as follows:

1. **Usage tracking:** Iterates through customer assignments; if a warehouse exceeds its capacity, the customer is flagged as unassigned.
2. **Reassignment:** for each unassigned customer:
    a. First searches open facilities with spare capacity and assigns the customer to the one with the lowest allocation cost.
    b. If none exist, searches closed facilities with sufficient capacity and opens the one minimizing the sum of fixed opening cost and customer allocation cost.

To demonstrate the process, we provide an example of this repair for the encoded individual given below.

```
[11, 6, 2, 2, 3, 13, 10, 10, 12, 8, 5, 10, 6, 7, 10, 12, 13, 10, 1, 0, 7, 2, 4, 7, 3, 1
3, 14, 9, 13, 9, 3, 3, 7, 12, 3, 7, 12, 13, 6, 14, 2, 6, 1, 7, 8, 11, 11, 2, 0, 13]
```

The feasibility repair checks every customer in order, upon checking the customer at index 36 it finds that assignment to facility 12 would exceed maximum capacity meaning reassignment is necessary. It then searches the opened facilities and picks the one with lowest allocation cost, in this case facility 5.

```
[11, 6, 2, 2, 3, 13, 10, 10, 12, 8, 5, 10, 6, 7, 10, 12, 13, 10, 1, 0, 7, 2, 4, 7, 3, 1
3, 14, 9, 13, 9, 3, 3, 7, 12, 3, 7, 5, 13, 6, 14, 2, 6, 1, 7, 8, 11, 11, 2, 0, 13]
```

### Genetic Operators

**Crossover:**
Uses uniform crossover. For each customer gene, there is a 50% chance of swapping assignments between parents, producing two offspring.
**Mutation:**
Iterates through each customer gene with probability `mutation_rate`. If triggered, it reassigns the customer to a different random warehouse.

Both operators are followed by `feasibility_repair` to make sure the solutions are valid.

### Objective Evaluation

The `evaluate_solution` function calculates both conflicting minimization objectives:

- **Facility Opening Cost($f_1$):** Finds unique facilities using `set(solution)` and sums their fixed opening costs from `warehouse_data`.
- **Customer Allocation Cost($f_2$):** Sums the assignment cost for each customer from `customer_data` based on their assigned facility.

## VEGA

VEGA is the oldest and simplest algorithm of the two. The design can be described through the `run()`method. It starts by initializing the population using `create_population()` from the base-class. An empty archive is initialized, where nondominated solutions will be kept and updated throughout all generations, introducing elitism.

For each generation, the current population is added to the archive. The `nondominated_solution()` method filters the archive for dominated solutions with the `dominates()` method. Individual A dominates individual B if (1): A is not worse than B in any of the objectives, and (2): A is better than B in at least one

objective. Only nondominated solutions are returned, and `unique_solutions()` removes duplicate solutions from the archive.

`VEGA-selection()` shuffles the population and splits it into two non-overlapping lists, before calculating weights for opening-cost and customer-cost based on the worst/most expensive individual in the population. This makes relative probabilities which are the ground for selection done by `random.choices()`. The lower the cost, the bigger the probability to be chosen. The selection is done with replacement, and at last, the two selected lists are put together, shuffled and returned as the mating-pool.

`reproduce()` takes mating-pool as input and makes the next generation. Two and two individuals from the mating-pool are used as parents, and children are made by using `crossover()` and `mutation()` from the base-algorithm. `feasibility-repair()` and `evaluate_solution()` is also done to assure valid solutions, before adding the children to the off-spring list. At last, the archive is filtered one last time, before the code-product returns a list with nondominated solutions from all the generations.

## NSGA-II

NSGA-II is an elitist multi-objective evolutionary algorithm designed to find a diverse Pareto-optimal front balancing both minimization problems. It guides convergence toward optimal trade-offs using fast non-dominated sorting while preserving solution diversity through crowding distance.

### Non-Dominated Sorting

The `non_dominated_sort` function splits the population into Pareto fronts based on their dominance across both objectives($f_1$ and $f_2$).

- Compares all individual pairs (`i, j`) to compute the domination count n (how many solutions dominate `i`) and the set S of solutions dominated by `i`.

- Solutions with `n = 0` are from the first Pareto front (`rank = 0`)
- Iteratively decrements the domination counts of solutions in `S` to extract following fronts (`rank = 1, 2, …`).

## Crowding Distance

The `calculate_crowding_distance` function preserves diversity along the trade-off surface by estimating local density:

- For each objective, sorts individuals within the front.
- Assigns boundary solutions an infinite distance.
- For intermediate solutions, adds the normalized distance between their neighboring solutions$(f_{k+1} - f_{k-1})/(f_{max} - f_{min})$.

## Tournament Selection

The `tournament_selection` function uses binary tournament selection (`tournament_size = 2`) with the crowded comparison operator:

- Randomly samples two individuals and compares them using (`ind.rank, -ind.crowding_distance`).
- Selects the candidate with the lower Pareto rank.
- If ranks are tied, selects the candidate with the larger crowding distance to favor less crowded regions.

## Main Loop

**Initialization:**

Calls `create_population()` to initialize `pop_size` feasible solutions and assigns initial ranks and crowding distances.

**Offspring Generation (`create_children`):**

Uses `tournament_selection` to select parents, applies crossover with probability `crossover_rate`, mutation with `mutation_rate`, and repairs all offspring with `feasibility_repair`.

**Elitist Replacement (`create_new_population`):**

Merges parents and offspring (*2N*), performs non-dominated sorting, and fills the next generation front by front. If the boundary front exceeds `pop_size`, individuals are sorted descending by crowding distance to retain only the most diverse solutions.

**Termination:**

Repeats for `generations` iterations and returns the final Pareto-ranked population.

## Differences in multi-objective selection

VEGA randomly partitions the population into *k* sub-groups, where *k* equals the number of objectives. Each subpopulation is evaluated and selected given its objective, before combining the selected individuals into a mating-pool and applying genetic operators.

NSGA-II ranks solutions using nondominated sorting. Replacement is elitist, combining parent and offspring generations to choose the next generation based on nondomination ranking. If the last front cannot be fully accommodated, NSGA-II uses crowding distance to fill the population while preserving diversity.

## Experiment methodology

For a fair comparison, VEGA and NSGA-II were evaluated using the same benchmark setup and a fixed random seed (`BASE_SEED = 42 + run_id`) to ensure reproducibility. Both algorithms run on all four parameter configurations (appendix A). Each configuration was run 10 times on the same datasets, giving both algorithms the same experimental budget and evaluation conditions. The four parameter sets provide a baseline and one set for each parameter change. This clearly shows how each adjustment affects convergence, diversity and runtime. For metrics we measure the amount of unique non-dominated solutions, time and the hypervolume. We calculate hypervolume using min-max normalization, the reference point (1.1, 1.1) is used to measure the dominated area.

## Comparison of algorithms across varying problems

Under the same budget, NSGA-II performs better on quality measures. It produces more non-dominated solutions and finds more diversity as seen in the table below. VEGA on the other hand consistently wins on speed for all problem sizes. This is the expected behavior for these algorithms and satisfactory behavior. NSGA-II is therefore more suitable for cases where solution quality is imperative while VEGA could be considered for cheap problem solving.

## Parameter configuration analysis

Full results are in the appendix.
Dou

### Decreased crossover

Decreasing crossover (0.8 -> 0.4) had little effect on runtime compared to baseline. The effect on both convergence and diversity was small, which might indicate that the processes of mutation and feasibility repair are very significant in the algorithm.

### Increased mutation

Increasing mutation (0.05 - 0.2) had a more significant impact. Convergence dropped significantly in both algorithms and diversity in NSGA-II results also dropped as a result of the increased randomness in the system.

In conclusion, NSGA-II consistently finds more non-dominated solutions and achieves higher hypervolume, but VEGA runs 2 - 3x faster. Increasing population size does improve results, but has diminishing returns.

# Appendix

## Appendix A – Experiment parameters

```python
BASE_SEED = 42
HV_REF = (1.1, 1.1)

# Run on available CPU cores - 1 or 1 if none are detected
cores = os.cpu_count()
NUM_WORKERS = cores - 1 if cores is not None and cores > 1 else 1

NUM_RUNS = 10
GENERATIONS = 200

CONFIGS = {
    "baseline": {
        "pop_size": 100,
        "generations": GENERATIONS,
        "crossover_rate": 0.8,
        "mutation_rate": 0.05,
    },
    "high_pop": {
        "pop_size": 200,
        "generations": GENERATIONS,
        "crossover_rate": 0.8,
        "mutation_rate": 0.05,
    },
    "low_crossover": {
        "pop_size": 100,
        "generations": GENERATIONS,
        "crossover_rate": 0.4,
        "mutation_rate": 0.05,
    },
    "high_mutation": {
        "pop_size": 100,
        "generations": GENERATIONS,
        "crossover_rate": 0.8,
        "mutation_rate": 0.2,
    },
}
```

## Appendix B – benchmark of cap61

| Instance | Config | Algo | HV mean | HV std | HV best | HV worst | ND mean | Time (s) |
|---|---|---|---|---|---|---|---|---|
| cap61 | baseline | NSGA-II | 0.964 | 0.051 | 1.066 | 0.865 | 7.9 | 0.92 |
| cap61 | baseline | VEGA | 0.602 | 0.035 | 0.641 | 0.516 | 4.1 | 0.34 |
| cap61 | high_pop | NSGA-II | 0.991 | 0.087 | 1.126 | 0.866 | 8.9 | 3.04 |
| cap61 | high_pop | VEGA | 0.612 | 0.032 | 0.681 | 0.562 | 4.5 | 0.73 |
| cap61 | low_crossover | NSGA-II | 0.927 | 0.078 | 1.025 | 0.757 | 7.5 | 0.87 |
| cap61 | low_crossover | VEGA | 0.557 | 0.038 | 0.594 | 0.502 | 4.4 | 0.33 |
| cap61 | high_mutation | NSGA-II | 0.704 | 0.033 | 0.767 | 0.672 | 5.7 | 1.06 |
| cap61 | high_mutation | VEGA | 0.384 | 0.017 | 0.421 | 0.364 | 3.9 | 0.45 |

## Appendix C – benchmark of cap62

| Instance | Config | Algo | HV mean | HV std | HV best | HV worst | ND mean | Time (s) |
|---|---|---|---|---|---|---|---|---|
| cap62 | baseline | NSGA-II | 1.076 | 0.064 | 1.182 | 0.950 | 8.1 | 2.19 |
| cap62 | baseline | VEGA | 0.719 | 0.043 | 0.814 | 0.641 | 4.3 | 0.86 |
| cap62 | high_pop | NSGA-II | 1.055 | 0.082 | 1.179 | 0.952 | 8.8 | 6.95 |
| cap62 | high_pop | VEGA | 0.714 | 0.030 | 0.748 | 0.634 | 4.0 | 1.54 |
| cap62 | low_crossover | NSGA-II | 1.060 | 0.052 | 1.165 | 0.948 | 7.8 | 2.10 |
| cap62 | low_crossover | VEGA | 0.662 | 0.043 | 0.707 | 0.591 | 4.1 | 0.75 |
| cap62 | high_mutation | NSGA-II | 0.807 | 0.047 | 0.890 | 0.767 | 5.4 | 2.41 |
| cap62 | high_mutation | VEGA | 0.518 | 0.041 | 0.600 | 0.460 | 4.5 | 0.93 |

## Appendix D – benchmark of cap101

| Instance | Config | Algo | HV mean | HV std | HV best | HV worst | ND mean | Time (s) |
|---|---|---|---|---|---|---|---|---|
| cap101 | baseline | NSGA-II | 1.060 | 0.051 | 1.160 | 0.972 | 11.0 | 2.11 |
| cap101 | baseline | VEGA | 0.599 | 0.017 | 0.629 | 0.560 | 4.9 | 0.91 |
| cap101 | high_pop | NSGA-II | 1.111 | 0.052 | 1.176 | 1.035 | 13.7 | 6.60 |
| cap101 | high_pop | VEGA | 0.598 | 0.034 | 0.670 | 0.544 | 5.1 | 1.65 |
| cap101 | low_crossover | NSGA-II | 1.091 | 0.046 | 1.157 | 1.028 | 10.9 | 2.03 |
| cap101 | low_crossover | VEGA | 0.588 | 0.033 | 0.626 | 0.524 | 5.1 | 0.80 |
| cap101 | high_mutation | NSGA-II | 0.753 | 0.031 | 0.805 | 0.714 | 6.2 | 2.31 |
| cap101 | high_mutation | VEGA | 0.437 | 0.012 | 0.459 | 0.414 | 5.7 | 0.92 |

## Appendix E – benchmark of cap102

| Instance | Config | Algo | HV mean | HV std | HV best | HV worst | ND mean | Time (s) |
|---|---|---|---|---|---|---|---|---|
| cap102 | baseline | NSGA-II | 0.985 | 0.047 | 1.081 | 0.906 | 10.9 | 0.95 |
| cap102 | baseline | VEGA | 0.511 | 0.029 | 0.571 | 0.468 | 5.0 | 0.38 |
| cap102 | high_pop | NSGA-II | 1.038 | 0.055 | 1.159 | 0.972 | 13.6 | 2.99 |
| cap102 | high_pop | VEGA | 0.519 | 0.029 | 0.571 | 0.463 | 6.0 | 0.73 |
| cap102 | low_crossover | NSGA-II | 0.990 | 0.051 | 1.080 | 0.903 | 10.4 | 0.91 |
| cap102 | low_crossover | VEGA | 0.490 | 0.020 | 0.517 | 0.460 | 6.2 | 0.38 |
| cap102 | high_mutation | NSGA-II | 0.684 | 0.031 | 0.733 | 0.634 | 7.0 | 1.02 |
| cap102 | high_mutation | VEGA | 0.367 | 0.024 | 0.397 | 0.333 | 5.4 | 0.42 |

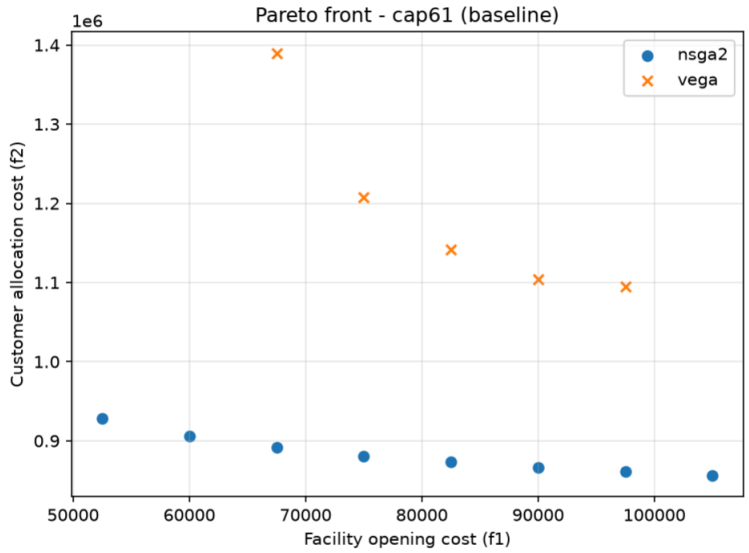
## Appendix F – benchmark of cap121

| Instance | Config | Algo | HV mean | HV std | HV best | HV worst | ND mean | Time (s) |
|---|---|---|---|---|---|---|---|---|
| cap121 | baseline | NSGA-II | 1.089 | 0.045 | 1.178 | 1.017 | 11.6 | 2.37 |
| cap121 | baseline | VEGA | 0.600 | 0.017 | 0.632 | 0.585 | 5.7 | 0.83 |
| cap121 | high_pop | NSGA-II | 1.122 | 0.040 | 1.188 | 1.068 | 15.0 | 6.92 |
| cap121 | high_pop | VEGA | 0.582 | 0.022 | 0.616 | 0.542 | 6.5 | 1.84 |
| cap121 | low_crossover | NSGA-II | 1.083 | 0.049 | 1.165 | 1.004 | 11.6 | 1.95 |
| cap121 | low_crossover | VEGA | 0.578 | 0.029 | 0.613 | 0.519 | 6.2 | 1.01 |
| cap121 | high_mutation | NSGA-II | 0.747 | 0.027 | 0.791 | 0.713 | 7.6 | 2.37 |
| cap121 | high_mutation | VEGA | 0.453 | 0.021 | 0.490 | 0.429 | 7.0 | 0.98 |

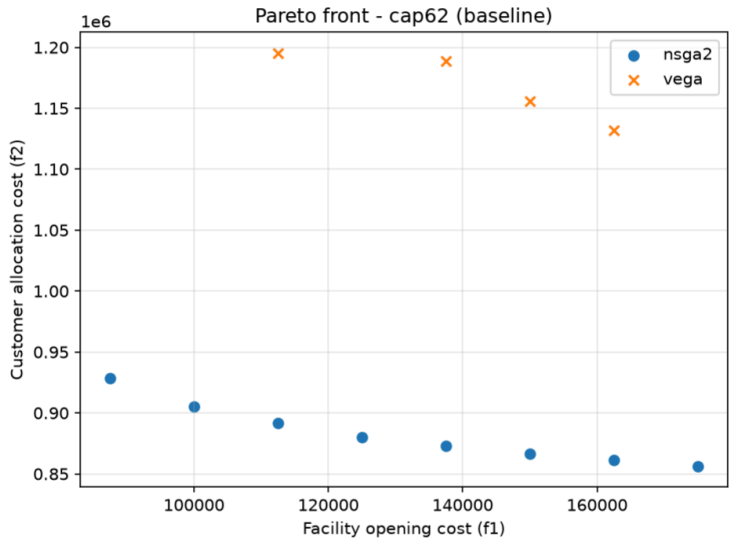
## Appendix G – benchmark of cap122

| Instance | Config | Algo | HV mean | HV std | HV best | HV worst | ND mean | Time (s) |
|---|---|---|---|---|---|---|---|---|
| cap122 | baseline | NSGA-II | 1.055 | 0.049 | 1.165 | 0.980 | 11.9 | 0.94 |
| cap122 | baseline | VEGA | 0.588 | 0.040 | 0.647 | 0.535 | 5.5 | 0.41 |
| cap122 | high_pop | NSGA-II | 1.100 | 0.045 | 1.147 | 1.023 | 15.1 | 3.13 |
| cap122 | high_pop | VEGA | 0.573 | 0.024 | 0.615 | 0.526 | 4.9 | 0.83 |
| cap122 | low_crossover | NSGA-II | 1.023 | 0.038 | 1.089 | 0.967 | 12.6 | 0.95 |
| cap122 | low_crossover | VEGA | 0.564 | 0.025 | 0.601 | 0.523 | 6.4 | 0.39 |
| cap122 | high_mutation | NSGA-II | 0.711 | 0.034 | 0.794 | 0.671 | 8.8 | 1.10 |
| cap122 | high_mutation | VEGA | 0.433 | 0.014 | 0.453 | 0.415 | 6.3 | 0.47 |

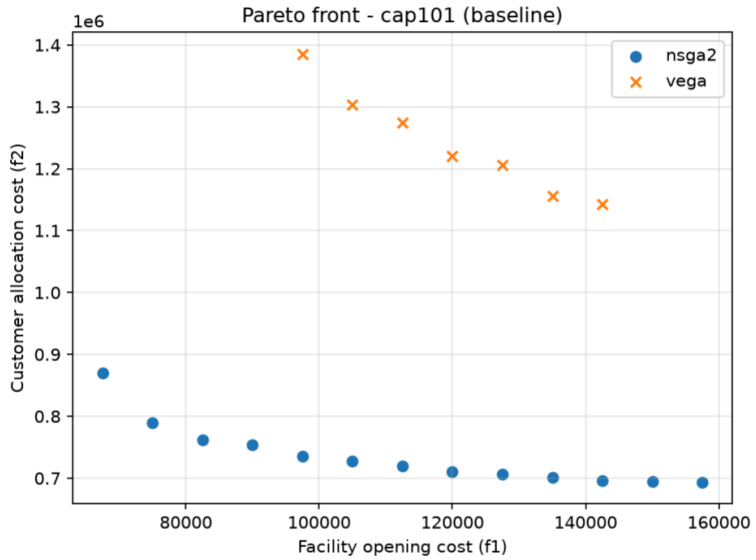
## Appendix H – pareto front cap61 (baseline)



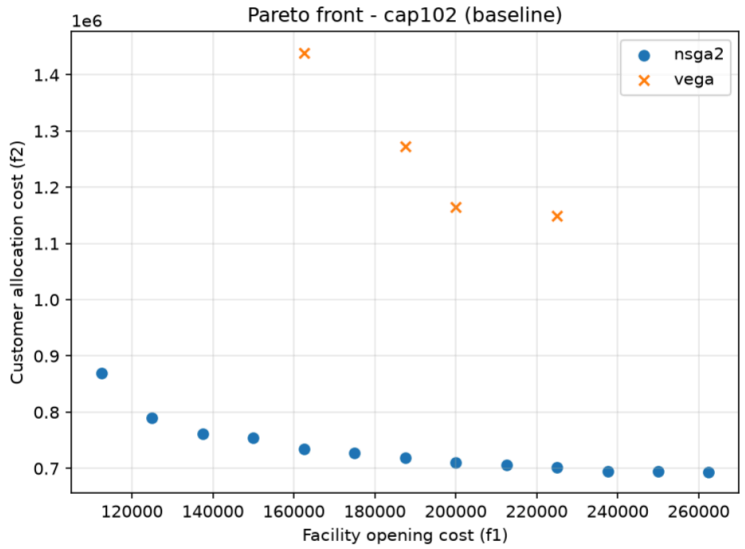

## Appendix I – pareto front cap62 (baseline)



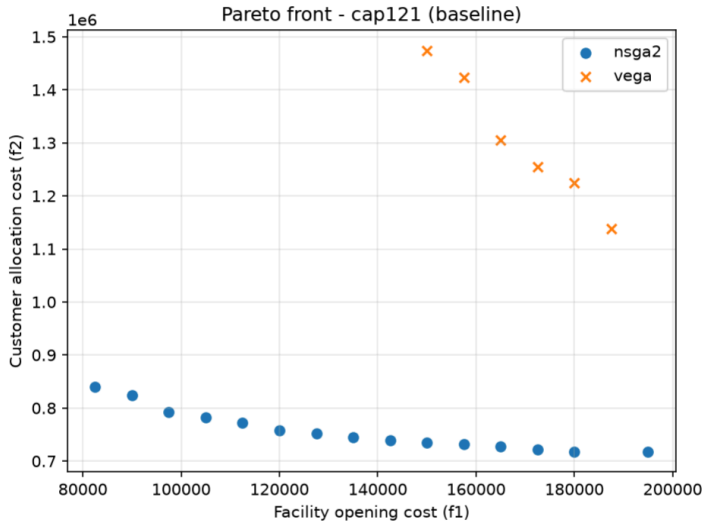
## Appendix J – pareto front cap101(baseline)




## Appendix K – pareto front cap102 (baseline)



## Appendix L – pareto front cap121(baseline)




## Appendix M – pareto front cap122 (baseline)

